Operational Test: Provenance Completeness as Standalone Procedure Specification in the Coordination Knowledge Substrate Pattern

Author: Wenxin Li (Independent Researcher)

ORCID: 0009-0004-8065-3235

Date: May 7, 2026

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its contribution is to formalize one of the operational tests CKS deployments must pass — the **provenance-completeness test** — as a standalone procedure specification with independent operational content, separable from the broader path-retraceability commitment of which it is one verifiable part. The test verifies the six provenance metadata fields the source paper requires of every piece of substrate content (§5), through a field-by-field inspection performed against actual substrate state.

Abstract

The CKS pattern requires every piece of substrate content to carry provenance — six metadata fields together sufficient to reconstruct who wrote the content, when, what was written, under what authorizing rule, through what executing cell, and linked to which specific cell execution. The requirement is stated once in the source paper (§5) and used across Claims 2 through 6 without restatement. Operationally, the requirement decomposes into a verification problem: given a deployed CKS substrate, can a reader confirm that every substrate change carries all six fields, correctly populated, and held in substrate-resident form? This note formalizes the verification problem as a standalone test with an explicit procedure, pass/fail criteria, list of anti-patterns the test detects, deployment-verification integration, and limits. The test is one of approximately sixteen Phase A5 operational tests in the derivation-note series; its scope is narrow by design, paired with the four-accountability-questions test that follows it, and complementary to the determinism and mediator-role tests that precede it.

1. Why the provenance-completeness test needs to be formalized as standalone

The provenance commitment is foundational. Path retraceability (§5), accountability under the plan/trace pair (§3.1), audit, reproducibility under composition, and orchestration retraceability across trigger and execution events all depend on the same underlying property: every substrate change carries enough metadata that it is fully addressable as substrate content.

The commitment is stated once and relied on many times. Operationally, however, "the substrate carries the right provenance" is not self-verifying. A deployed substrate may carry partial provenance, may aggregate multiple changes into one entry, may use vendor logs whose schemas do not map to the six fields, or may hold provenance in tools the substrate does not control. Each failure mode produces a system that *appears* compliant from the outside while violating the commitment in a specific, identifiable way. A separate, named, standalone test for provenance completeness — one that inspects the substrate field by field against actual substrate state — is what makes those failure modes detectable as such, rather than catchable only after they cause a downstream failure of accountability or reproducibility.

The standalone framing is also deliberate in its narrow scope. The four-accountability-questions test (the next note in this cluster) verifies that the four governance questions Naja and colleagues' accountability-plan vocabulary requires — who, when, under what authority, through what mechanism — can in fact be answered from the recorded provenance. The provenance-completeness test, formalized here, verifies that the underlying six fields are present, correctly populated, and substrate-resident. The two tests form a tight pair: field presence and correctness (this test) are necessary for question answerability (next test); answerability is what the fields exist to support.

The test is the eighth in the Phase A5 cluster covering substrate-state and AI-mediation commitments. The seven preceding tests cover the four governance rights, the mediator role, cell-behavior determinism, and read determinism. The next test covers the four accountability questions; subsequent Phase A5 notes cover source-of-truth, conflict-as-first-class, and other commitments through approximately sixteen tests in total.

2. The architectural commitment under test

The commitment is that every substrate change records all six provenance metadata fields, individually present and correctly populated, held in the substrate itself rather than in vendor systems external to it. The six fields are:

Field 1 — Who (actor identification). The actor that produced the change. For human-initiated changes, the actor is the identified human exercising the inspect-modify-override authority human-governed substrates preserve at all times. For LLM-mediated changes, the actor is the LLM, recorded with attribution to the cell and rule under which it operated per the AI-as-substrate-mediator commitment's recording requirement; the human authority remains governing, but the executing actor is the LLM.

Field 2 — When (timestamp). The time at which the change took effect against substrate state. The timestamp is what makes any path through the substrate orderable; without it, addressability of changes against time is lost.

Field 3 — What (change content). The substrate content that was added, modified, or removed. The content is what the change consists of, recorded at the granularity of the substrate's addressable units.

Field 4 — Why (authorizing rule). The rule under which the change was authorized — an orchestration rule for cell-mediated changes, an exercise of override authority for direct human changes. The authorizing rule is itself substrate-resident content the provenance entry references by addressable identifier, not a free-text rationale and not an external policy reference.

Field 5 — How (executing cell or mechanism). The cell that executed the change, or the direct-write mechanism by which a human exercised override authority. The cell is referenced by addressable identifier resolving to a substrate-resident cell definition.

Field 6 — Cell-execution-id (link to specific execution). The identifier of the specific cell execution that produced the change, distinguishing this execution from other executions of the same cell under the same rule. Without field 6, a change can be attributed to a cell and rule but not to a particular execution; with it, the change becomes addressable at the granularity of execution events, which is what orchestration retraceability and reproducibility under composition require.

The commitment further requires that all six fields live in substrate-resident form. Provenance held in vendor audit logs, tool-specific event streams, or external observability systems may be informationally adequate but is architecturally non-conformant: the substrate-only-paths commitment requires the trace to live where the rest of the substrate lives, or the substrate ceases to be the source of truth for what was decided and why.

3. The test procedure

The procedure is a field-by-field inspection performed against actual substrate state. The sequence is:

Step 1 — Trigger a substrate change. Select a cell expected to write to the substrate, execute it under a known orchestration rule, and observe the resulting substrate change. Repeat for each change type the deployment supports: direct human modification under preserved override authority, direct override action (which may require no orchestration rule per the no-justification property of override), AI-mediated cell processing, and orchestration trigger event that initiates cell execution and is itself substrate-recordable.

Step 2 — Inspect the resulting provenance record. Read the provenance entry the substrate produced for the change, directly, without LLM intermediation as a precondition. The provenance must be inspectable in the same form the substrate carries the rest of its content.

Step 3 — Verify all six fields are present. Confirm that fields 1 through 6 are individually present in the entry. A missing field at this step fails the test for partial-provenance regardless of how informative the fields that are present may be.

Step 4 — Verify each field is correctly populated. Field 1's actor identifier resolves to a specific identified human or to an LLM with attribution to the cell and rule under which it operated. Field 2's timestamp matches the time the change took effect. Field 3's content matches the change actually made. Field 4's rule reference resolves to a substrate-resident orchestration rule with an addressable identifier. Field 5's cell reference resolves to a substrate-resident cell definition. Field 6's execution-id is unique and resolves to a specific cell execution that the substrate also records as a first-class addressable event.

Step 5 — Verify substrate-residence. Confirm that the provenance entry lives in the substrate, not in a vendor audit log, a tool-specific event stream, or any external observability system. If the only source carrying any of the six fields is external, the test fails on substrate-residence regardless of whether the field's information is present somewhere.

Step 6 — Repeat across change types. Run steps 1 through 5 for each of: human modifications under preserved authority, override actions, AI-mediated cell processing, and orchestration trigger events. Provenance completeness is not a property of one change type; it is a property of every change type the deployment produces.

The test is repeatable, mechanical, and decidable: each field's presence and correctness is a yes/no determination, and the substrate-residence check is a yes/no determination on each field's location.

4. Test outputs

The test produces a pass/fail result with explicit criteria.

Pass. All six fields are present in every provenance entry inspected; each field is correctly populated against the actual change; the entries are substrate-resident; the test was performed across all the change types the deployment supports.

Fail. Any of the following conditions hold for any inspected entry: a field is missing; a field is present but does not resolve correctly (an actor identifier with no matching identity, a rule reference pointing to no substrate-resident rule, a cell reference pointing to no cell definition, an execution-id with no corresponding execution event); the provenance entry lives outside the substrate; the field schema records information that does not map to the six fields named here; the test was not run across all change types.

A pass on this test does not imply the broader retraceability commitment is satisfied — the four-accountability-questions test verifies that the questions the fields exist to answer can in fact be answered from them. A fail on this test does imply the retraceability commitment is violated, since field-level satisfaction is necessary for question-level satisfaction.

5. Anti-patterns the test detects

The test detects six categories of provenance violation.

Non-addressable writes (canonical anti-pattern). A change is written to the substrate without producing a provenance entry whose six fields are individually addressable. The change took effect, but the substrate cannot be queried about who made it, when, under what authority, through what mechanism, or in connection with which execution. The test fails on missing fields. This is the canonical violation of the retraceability commitment and the most common failure mode in deployments that treat provenance as an afterthought rather than as a substrate-schema requirement.

External-tool provenance. The change took effect against substrate state, but the only record of who made it lives in a vendor audit log, an observability platform, or a tool-specific event stream that the substrate does not own. The fields may be informationally complete; their location violates the substrate-only-paths commitment, and the test fails on substrate-residence. This failure mode makes vendor migration destroy retraceability, because the records do not move with the substrate.

Aggregated provenance. Multiple individual substrate changes are conflated into a single provenance entry — a daily summary, a session-level aggregation, a transactional batch — that records collective metadata but loses per-change addressability. The substrate cannot answer "who made *this* change" because the entry's granularity is coarser than the change. The test fails because each individual change does not have its own provenance.

Vendor audit logs not mapping to the six fields. Some vendors offer audit logging whose schema records timestamps and actor identifiers but not authorizing-rule references, executing-cell references, or cell-execution-ids — or records all six conceptually but with field semantics that do not map to substrate-resident content. The actor field exists but does not resolve to a substrate-recorded identity, or the rule field exists but does not reference a substrate-resident rule. The test fails on field-population correctness.

Missing cell-execution-id linking. Field 6 is treated as derivable from fields 1 through 5 rather than as an independent identifier. The execution-id may be absent, may be aliased to the cell identifier (in which case repeated executions of the same cell are indistinguishable), or may be derived from the timestamp (in which case timestamp collisions destroy uniqueness). The test fails on field 6's presence or uniqueness, and the failure breaks orchestration retraceability across trigger and execution events specifically.

Partial provenance. The schema records some of the six fields and not others — typically fields 1, 2, and 3 (who, when, what), with fields 4, 5, and 6 (why, how, execution-id) treated as optional or as cell-internal state. The fields present may be correctly populated, but the test fails on completeness, because the four accountability questions cannot be answered without all six.

The field-by-field structure of the procedure is what makes each anti-pattern nameable as a distinct failure rather than collapsing them into a generic "provenance is bad."

6. Integration with deployment verification

The test is intended as a recurring verification step, not a one-time gate. Five integration points apply.

Initial deployment validation. Before activation for production use, the test is run end-to-end against the deployed substrate to verify that provenance generation works as designed for every change type the deployment supports.

Cell-architecture-change verification. When cell logic changes — a new cell type, a revised orchestration rule, altered substrate-write semantics — the test is rerun against the affected cells to verify that provenance generation still produces complete entries.

Orchestration-change verification. When the workflow engine that triggers cell execution changes — a new trigger pattern, a revised trigger-event schema, a replacement engine — the test is rerun to verify that orchestration trigger events are still recorded as substrate provenance entries distinct from execution events.

Composition-partner verification. When the substrate is composed with another substrate or with an adjacent component, the test is rerun across the composition boundary to verify that provenance is preserved across the composition rather than degraded at the boundary.

Vendor-migration verification. When the host environment changes — a different commodity tool, a different LLM vendor, a different storage backend — the test is rerun to verify that the migration carried the substrate's provenance entries with it and that post-migration changes generate complete provenance under the new host.

The test's repeatability and field-by-field structure are what make it suitable for these integrations: each integration point reduces to "run the test, inspect the result," with no per-integration redesign of the verification procedure.

7. Limits of the test

The test verifies provenance completeness specifically. The scope of "specifically" is worth stating explicitly.

It does not verify accountability question answerability. The four-accountability-questions test verifies that the questions the fields exist to answer can be answered from the provenance. Field presence and correctness are necessary for question answerability but not sufficient: a deployment whose fields are individually correct may still have rule references that resolve only to vague rules, cell references that resolve to cells whose definitions are themselves underspecified, or execution-ids whose corresponding executions are not addressable as substrate events.

It does not verify determinism. The cell-behavior-determinism test and the read-determinism test cover that ground. A non-deterministic cell may still produce complete provenance about each of its (varying) executions, and a deterministic cell may produce incomplete provenance if its provenance-emission code has bugs.

It does not verify the mediator role. The mediator-role test verifies that the LLM operates under the five mediator properties. Provenance completeness verifies that LLM-mediated changes are recorded with attribution per the recording property, but does not verify that the LLM is operating under the other four properties.

It does not verify provenance content correctness. The test verifies that fields are present, populated, and substrate-resident — not that the recorded content is "true" or that the recorded change was "correct." A fabricated provenance entry that satisfies the schema is detectable only at substantive audit, not by this test.

The narrow scoping is the test's value. A deployment that passes this test has a verified property — the substrate's provenance schema is being satisfied for every change — that a deployment passing only a generic "audit" check does not.

8. The test, in one sentence

A CKS deployment passes the provenance-completeness test if and only if every substrate change, across every supported change type, generates a substrate-resident provenance entry whose six fields (actor, timestamp, change content, authorizing rule, executing cell or mechanism, cell-execution-id) are individually present, correctly populated, and resolved to substrate-resident referents.

9. Why naming the test as standalone matters

Treating provenance verification as one composite check inside a broader audit procedure obscures the field-by-field structure of the failure modes. A composite check fails opaquely — "the audit didn't pass" — and downstream remediation is undirected. The standalone framing, with field-by-field steps and named anti-patterns, makes each failure mode addressable: a missing field 6 is a different remediation from a vendor-resident audit log, which is a different remediation from aggregated entries, which is a different remediation from a vendor schema that does not map. Without the standalone framing, deployments that pass surface audits while violating one or more of these specific failure modes remain in the field undetected.

The continuing AI-mediation-cluster framing matters for the same reason. Provenance completeness, accountability question answerability, mediator role, cell-behavior determinism, and read determinism are five distinct properties a deployment must satisfy to be CKS-coherent on the substrate-state-and-mediation axis. Each has its own standalone test, and the tests' independence — pass/fail decidable for each on its own — is what makes the full-stack verification tractable. The pair-structure with the four-accountability-questions test is the local instance: this test verifies the fields the questions are answered from; the next verifies the questions can in fact be answered. Either could pass while the other failed.

A deployment that passes this test has a verified property that a deployment passing only a generic audit does not. Subsequent work that reuses the test specification, adapts it for different host environments, or composes it with other CKS deployment-verification steps should use "the provenance-completeness test" in the sense formalized here. Subsequent work that uses the term differently is using a different test, and the difference should be named.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems.* April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *Operational Test: Provenance Completeness as Standalone Procedure Specification in the Coordination Knowledge Substrate Pattern.* May 7, 2026. ORCID: 0009-0004-8065-3235.